SHRI SHAHAJI CHHATRAPATI MAHAVIDYALAYA, DASARA CHOWK, KOLHAPUR

ACADEMIC PROJECT REPORT ON,

"STUDENT EXPENSE TRACKER AND ANALYZER"

SUBMITTED TO,
SHIVAJI UNIVERSITY, KOLHAPUR
FOR THE DEGREE OF
BACHELOR OF COMPUTER APPLICATIONS (BCA)

SUBMITTED BY,

Name: _____
Seat Number: -
PRN Number : -

UNDER THE GUIDANCE OF,

Asst. Prof. Miss. Swati B. Patil
Department of Computer Applications

THROUGH
THE PRINCIPAL
PROF. DR. R. K. SHANEDIWAN
SHRI SHAHAJI CHHATRAPATI MAHAVIDYALAYA, KOLHAPUR
ACADEMIC YEAR 2026-27

Shri Shahu Chhatrapati Shikshan Sanstha's
Shri Shahaji Chhatrapati Mahavidyalaya, Kolhapur
Department of BCA

CERTIFICATE OF PROJECT COMPLETION

This is to certify that _____, student of Bachelor of Computer Applications (BCA) from Shri Shahaji Chhatrapati Mahavidyalaya, Kolhapur, has successfully completed the major academic project entitled "Student Expense Tracker and Analyzer" in partial fulfillment of the requirements for the award of Bachelor of Computer Applications (BCA) degree by Shivaji University, Kolhapur.

The academic project work was conducted during the academic semester of 2026-27, during which the student designed, engineered, tested, and demonstrated a full-stack personal finance tracking platform utilizing Python Flask, Streamlit, MySQL Database, and Pandas analytical engines.

Throughout the project tenure, the student demonstrated genuine dedication, punctuality, and technical diligence towards system architecture and research objectives. The student successfully fulfilled all prescribed university guidelines and project deliverables under our continuous guidance.

Place: Kolhapur
Date:

Project Guide **External Examiner** **BCA Coordinator**

Shri Shahu Chhatrapati Shikshan Sanstha's
Shri Shahaji Chhatrapati Mahavidyalaya, Kolhapur
Department of BCA

DECLARATION

I hereby declare that the academic project report entitled "Student Expense Tracker and Analyzer" submitted to Shivaji University, Kolhapur, in partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Applications (BCA), is a record of original software development work carried out by me under the guidance of Asst. Prof. Miss. Swati B. Patil, Department of BCA, Shri Shahaji Chhatrapati Mahavidyalaya, Kolhapur.

I confirm that the software implementation and documentation presented in this report represent authentic work completed by me and have not been submitted previously to any other university or academic board for the award of any diploma, degree, or fellowship.

All sources of literature, frameworks, algorithms, and documentation references utilized in this project have been acknowledged accurately.

I accept full personal responsibility for the originality and authentic execution of the software system and technical contents presented in this deliverable.

Student Name: _____
Seat number:
Course: Bachelor of Computer Applications (BCA)
Date:
Place: Kolhapur

Signature: _____

Shri Shahu Chhatrapati Shikshan Sanstha's
Shri Shahaji Chhatrapati Mahavidyalaya, Kolhapur
Department of BCA

ACKNOWLEDGEMENT

I sincerely express my profound gratitude to Shri Shahaji Chhatrapati Mahavidyalaya, Kolhapur, and the Department of Computer Applications (BCA) for providing me with the essential infrastructure, computational laboratories, and intellectual environment necessary to undertake and successfully complete this project.

I extend my deepest gratitude to my esteemed project guide, Asst. Prof. Miss. Swati B. Patil, for her indispensable technical suggestions, meticulous review, continuous motivation, and constructive criticism throughout each software development sprint.

I respectfully express my sincere regards to our respected Principal, Prof. Dr. R. K. Shanediwan, for his administrative leadership, institutional support, and persistent encouragement towards technical research and software innovation.

I also express my heartfelt appreciation to the internal and external examiners appointed by Shivaji University, Kolhapur, for their objective assessment, valuable evaluation, and constructive guidance during project review sessions.

Finally, I am deeply indebted to my parents, family members, faculty mentors, and friends whose relentless moral encouragement, patience, and assistance enabled me to successfully complete this academic endeavor.

Student Name: _____
Seat number:
Course: Bachelor of Computer Applications (BCA)
Date:
Place: Kolhapur

INDEX

S.N.	Chapter Title & Topical Sections	Page No.
1.	Chapter 1: Introduction • Background of the Problem • Existing System and its Limitations • Objective of the Project • Scope of the Project	1
2.	Chapter 2: System Study / Proposed System Objectives • Proposed System Objectives • Software Development Model (Agile Iterative Model) • Feasibility Study (Technical, Operational, Economic)	5
3.	Chapter 3: Requirement Analysis • Functional Requirements • Non-Functional Requirements • Hardware & Software Requirements	8
4.	Chapter 4: System Design • System Architecture Diagram • Data Flow Diagrams (Level 0 & Level 1) • UML Diagrams (Use Case, Class, Sequence) • Database Design (ER Diagram & Data Dictionary)	12
5.	Chapter 5: Implementation • Implementation Architecture & Environment • Module-wise Screenshots of Input & Output Design • Financial Reports Generated • Code Snippets (Important Parts Only)	20
6.	Chapter 6: Testing • Testing Methodology • Automated Test Cases & Verification Results Table	32
7.	Chapter 7: Conclusion & Future Scope • Summary of Work • System Limitations • Learning Outcomes • Possible Enhancements • References (IEEE / APA Format) • Appendices (API Catalog, Seed Data, File Tree, Meet Record)	36

CHAPTER 1

INTRODUCTION

Chapter 1: Introduction

1.1 Background of the Problem

Financial literacy and personal budgeting represent crucial life competencies that undergraduate and postgraduate college students must cultivate during their tertiary education. While residing in college hostels, rental private paying-guest (PG) accommodations, or commuting daily to educational institutions, students receive periodic monetary allowances, academic scholarships, gig-economy income, or family allowances. Simultaneously, students incur a multitude of non-negotiable living expenditures including mess dining charges, textbook purchases, stationery materials, local public transit fares, mobile cellular recharges, semester laboratory fees, and leisure outings.

In the absence of a structured, intuitive tracking tool, the majority of college students struggle with persistent financial disorganization. Students frequently experience severe mid-month and month-end cash-flow deficits, wondering how their monetary inflows evaporated. Analog record-keeping methods—such as pocket spiral diaries, physical paper receipts, mobile notes apps, or generic spreadsheets—suffer from high attrition rates due to manual data entry friction, zero automated budget alerts, lack of historical comparisons, and absence of visual financial feedback.

1.2 Existing System and Its Limitations

In existing academic environments, college students primarily track expenses using three traditional methods: manual physical paper notebooks, unstructured note-taking applications on smartphones, or complex commercial corporate accounting suites. Detailed empirical analysis reveals critical structural limitations across these existing practices:

- **Manual Paper Diaries & Spiral Ledgers:** Physical paper tracking is highly susceptible to physical loss, ink smudging, and water damage. It requires cumbersome manual arithmetic calculations with a pocket calculator, resulting in mathematical errors and providing zero visual analytical charts or automated threshold notifications.
- **Unstructured Mobile Notepad Notes:** Typing casual transaction notes in mobile smartphone notepads lacks relational database structure. Transactions cannot be filtered by date ranges or sorted by expenditure categories, making comprehensive month-over-month audits virtually impossible.
- **Complex Commercial Banking & Enterprise Accounting Tools:** Commercial enterprise software (e.g., Tally, QuickBooks) and commercial mobile banking aggregators are overly complicated for student needs. They demand double-entry bookkeeping knowledge, require invasive bank account SMS scraping, exhibit

privacy risks, or present distracting corporate advertisements and subscription paywalls.
- **Absence of Proactive Category Budget Guardrails:** Existing simple trackers merely log transactions retrospectively after funds are spent, failing to provide real-time warning indicators when spending approaches 80% or exceeds category-specific limits.

1.3 Objective of the Project

The overarching objective of this academic software engineering project is to design, implement, test, and deliver a dependable, localized, full-stack personal finance tracking platform entitled “Student Expense Tracker and Analyzer”. The concrete software engineering objectives encompass:

- **1. Dual-Stream Financial Ledger:** Enable students to seamlessly record monetary inflows (allowances, scholarships, freelance gigs) and categorized outflows (food, transport, education, shopping) with decimal numeric precision.
- **2. Decoupled 3-Tier Architecture:** Implement a robust client-server separation where a modern Streamlit user interface communicates via HTTP JSON requests with a Python Flask REST API, backed by a persistent MySQL relational database.
- **3. Category-Specific Monthly Budgeting:** Empower students to establish spending caps for individual expense categories for any calendar month and calculate real-time utilization percentages with configurable 80% warning indicators.
- **4. Interactive Analytical Visualizations:** Generate dynamic 6-month and 12-month Income vs. Expense bar charts, category distribution breakdowns, and month-over-month comparisons using Pandas and Altair.
- **5. Automated Statement CSV Export:** Allow students to generate filtered financial transaction statements and export official CSV reports for scholarship documentation, parental reporting, or academic audit.
- **6. Offline Localhost Privacy:** Guarantee 100% data privacy and zero dependence on commercial external banking APIs or paid cloud infrastructure by running completely on localhost.

1.4 Scope of the Project

The technical scope of the project encompasses a complete software solution engineered for local student computing environments:

- **Frontend User Interface Tier:** A responsive, component-driven web application built with Streamlit, incorporating modern cards, status badges, dynamic forms, and interactive visualization charts.

- **Backend Application Tier:** A modular RESTful microservice built on Python Flask 3.x, structured with blueprint routes, dedicated business logic services, validation utilities, and CORS middleware.
- **Database Persistence Tier:** A normalized relational database schema engineered on MySQL Server 8.0, utilizing foreign key constraints, indexes, timestamps, and DECIMAL data types.
- **Operational Boundary:** The application strictly operates as a local personal finance tracking system based on entered transaction records. It does not integrate with live bank APIs or process online credit card transactions, ensuring zero security vulnerabilities.

CHAPTER 2

SYSTEM STUDY / PROPOSED SYSTEM

Chapter 2: System Study / Proposed System Objectives

2.1 Proposed System Objectives

The proposed Student Expense Tracker and Analyzer resolves the operational drawbacks of analog diaries and complex commercial tools by delivering an intuitive, real-time financial tracking workflow. Key architectural advantages include:

- **Instant Financial Feedback:** A centralized dashboard displays Total Income, Total Expenses, Net Recorded Balance, and Budget Utilization with immediate automatic recalculation upon every transaction entry.
- **Relational Category Integrity:** Transaction forms dynamically populate categories according to the selected transaction type (Income vs. Expense), preventing classification errors.
- **Proactive Budget Guardrails:** Color-coded visual alerts (Safe, Warning at 80%, Exceeded at >100%) alert students prior to exhausting their monthly funds.
- **Graceful API Offline Handling:** The Streamlit frontend incorporates intelligent network resilience, presenting clear recovery instructions if the Flask backend server is temporarily paused.

2.2 Software Development Model

To ensure disciplined software engineering, rapid milestone verification, and systematic quality assurance, the **Agile Iterative Model** was chosen as the governing Software Development Life Cycle (SDLC) framework. The project was decomposed into five progressive sprints:

- **Sprint 1 — Database Architecture & Schema Initialization:** Designed normalized relational schema (`users`, `categories`, `transactions`, `budgets`) under `DataBase/schema.sql`, configured Flask-SQLAlchemy with PyMySQL, and created automated migration/seeder scripts.
- **Sprint 2 — RESTful Backend API & Business Services:** Engineered Flask REST endpoints under `Routes/` and business validation services under `Services/` for transaction CRUD, category validation, monthly budget calculations, and dashboard summaries.
- **Sprint 3 — Streamlit Frontend Views & Component Assembly:** Developed responsive UI views under `frontend_views/` for Dashboard, Transaction History, Add Transaction, Monthly Budgets, Spending Analytics, Category Management, and Reports.
- **Sprint 4 — Data Visualization, Analytics & CSV Export:** Integrated Altair charts and Pandas dataframes to render 6-month trends, category breakdowns, month-over-month comparisons, and CSV export functionality.

- **Sprint 5 — Automated Pytest Suite & Academic Documentation:** Formulated automated unit, integration, and financial calculation tests with pytest, verified zero warnings, and authored the comprehensive college project documentation.

2.3 Feasibility Study

Prior to technical implementation, a rigorous three-dimensional feasibility study was conducted to confirm the viability of the project:

- **1. Technical Feasibility:** The project utilizes mature, industry-standard technologies: Python 3.10+, Flask, Streamlit, MySQL Server, and SQLAlchemy. Python provides expressive syntax and robust data processing libraries (Pandas). Flask offers lightweight, flexible routing with minimal overhead. Streamlit provides high-performance, reactive UI state management. MySQL delivers ACID-compliant transaction persistence. All libraries are open-source, well-documented, and fully compatible with standard computer systems. Hence, the project is technically feasible.
- **2. Operational Feasibility:** The application interface is purposefully tailored for non-technical college students. Form inputs feature intuitive controls (date pickers, dropdown selects, numeric steppers), and tables support instant keyword search. The system requires zero complex database commands from the user and operates seamlessly on any desktop web browser. Hence, operational feasibility is exceptionally high.
- **3. Economic Feasibility:** The entire software ecosystem comprises 100% free, open-source software tools with zero commercial licensing fees. Hardware requirements are satisfied by standard personal computers or college lab machines. Running locally eliminates costly cloud hosting, database SaaS subscriptions, or paid payment gateway integrations. Hence, the project is economically advantageous.

CHAPTER 3

REQUIREMENT ANALYSIS FUNCTIONAL

Chapter 3: Requirement Analysis

3.1 Functional Requirements

Functional requirements delineate the precise operational capabilities, business rules, and user interaction responses implemented within the Student Expense Tracker and Analyzer:

Req ID	Module	Functional Requirement Description
FR-01	Financial Dashboard	Calculate and display Total Income, Total Expenses, and Net Balance for the selected calendar month.
FR-02	Financial Dashboard	Display overall category budget utilization percentage and highlight the highest-spending category.
FR-03	Financial Dashboard	Render interactive 6-month Income vs. Expense bar charts and category distribution charts.
FR-04	Transaction Entry	Record income and expense transactions with amount (>0), valid date, payment method, category, and note.
FR-05	Dynamic Categorization	Dynamically filter category dropdown based on transaction type (Income vs. Expense) to prevent invalid links.
FR-06	Transaction Search	Filter transaction history by date range, transaction type, category, and keyword search across notes/methods.
FR-07	Transaction Mutation	Enable updating existing transaction fields and permanently deleting records via authenticated API calls.
FR-08	Budget Management	Set monthly spending limits for expense categories and reject duplicate budgets for the same month/year.
FR-09	Budget Alert Engine	Compute real-time category expenses against budgets and trigger visual warning alerts when spending ≥80%.
FR-10	Spending Analytics	Analyze 12-month financial flows, compute average monthly expenses, and evaluate month-over-month differences.
FR-11	Category System	Pre-seed standard categories, support custom category creation, and block unsafe deletion of in-use categories.
FR-12	Report Export	Generate period financial statement summaries and export complete transaction ledgers into standardized CSV files.

3.2 Non-Functional Requirements

Non-functional requirements specify performance benchmarks, operational constraints, and quality attributes essential for reliable academic execution:

- **Performance & Response Time:** API endpoint response times execute under 50 milliseconds for typical queries on localhost. Streamlit dashboard refreshes within 300 milliseconds.
- **Data Integrity & Precision:** All monetary figures are stored using MySQL `DECIMAL(10,2)` rather than floating-point representations to eliminate floating-point arithmetic rounding discrepancies.
- **Relational Consistency:** Foreign key constraints (`ON DELETE RESTRICT` for categories with transactions, `ON DELETE CASCADE` for budgets) guarantee referential integrity.
- **Security & Input Sanitization:** All database operations employ parameterized SQL queries via SQLAlchemy ORM, eliminating SQL injection risks. Secrets are kept out of source code via `.env` files.
- **Resilience & Error Handling:** The client gracefully catches HTTP timeout or connection errors, presenting clear recovery advice without crashing or dumping raw stack traces.
- **Cross-Platform Portability:** The application runs smoothly across Windows, macOS, and Linux operating systems running Python 3.10+ and standard web browsers.

3.3 Hardware & Software Requirements

The minimal and recommended system configurations for deploying and operating the system are summarized below:

Component	Minimum Requirement	Recommended Specification
Processor (CPU)	Intel Core i3 / AMD Ryzen 3 (2.0 GHz)	Intel Core i5 / AMD Ryzen 5 or higher
System Memory (RAM)	4 GB DDR3 / DDR4	8 GB / 16 GB DDR4 / DDR5
Storage Space	500 MB free hard disk / SSD space	2 GB SSD free space for logs and data
Display Resolution	1280 x 720 (HD)	1920 x 1080 (Full HD) display
Input Devices	Standard Keyboard & Mouse / Trackpad	Keyboard & Multi-gesture Mouse

Component	Technology / Software Package	Operational Purpose / Role
Operating System	Microsoft Windows 10 / 11 (64-bit)	Host environment for execution and development
Runtime Environment	Python 3.10+ / Python 3.14 (64-bit)	Core programming runtime and interpreter
Backend Web Framework	Flask 3.x	RESTful API creation, routing namespaces, and CORS
Frontend UI Framework	Streamlit 1.30+ / 1.58.0	Reactive user interface and dashboard engine
Relational Database	MySQL Server 8.0+ / MariaDB	ACID-compliant relational persistence on port 3306
Database Driver & ORM	SQLAlchemy 2.0+ / PyMySQL 1.2+	Object Relational Mapping and parameterized queries
Data Processing	Pandas 2.0+ / 3.0.3	Dataframe manipulation, aggregations, and CSV export
Data Visualization	Altair 5.0+ / Streamlit Charts	Interactive bar charts, trends, and distribution graphs
Automated Testing	Pytest 8.0+ / 9.1.1	Automated unit, API, and financial calculation testing

CHAPTER 4

SYSTEM DESIGN

Chapter 4: System Design

4.1 System Architecture Diagram

System design translates abstract functional requirements into technical engineering blueprints. The Student Expense Tracker and Analyzer implements an enterprise-grade **3-Tier Layered Architecture** consisting of the Presentation Layer, the Application/Service Layer, and the Data Persistence Layer.

- **1. Presentation Layer (Streamlit UI - Port 8501):** Renders the user interface, manages session state, captures user form inputs, triggers client-side validation, and formats numerical figures into Indian Rupee (₹) currency tokens.
- **2. Application & API Layer (Flask REST API - Port 5000):** Exposes RESTful endpoints (`/api/transactions`, `/api/categories`, `/api/budgets`, `/api/dashboard`, `/api/analytics`), validates payloads, enforces business logic, and computes monthly financial aggregations.
- **3. Data Persistence Layer (MySQL Database - Port 3306):** Maintains relational schema integrity across normalized tables, executing parameterized SQL queries via SQLAlchemy ORM with connection pooling.

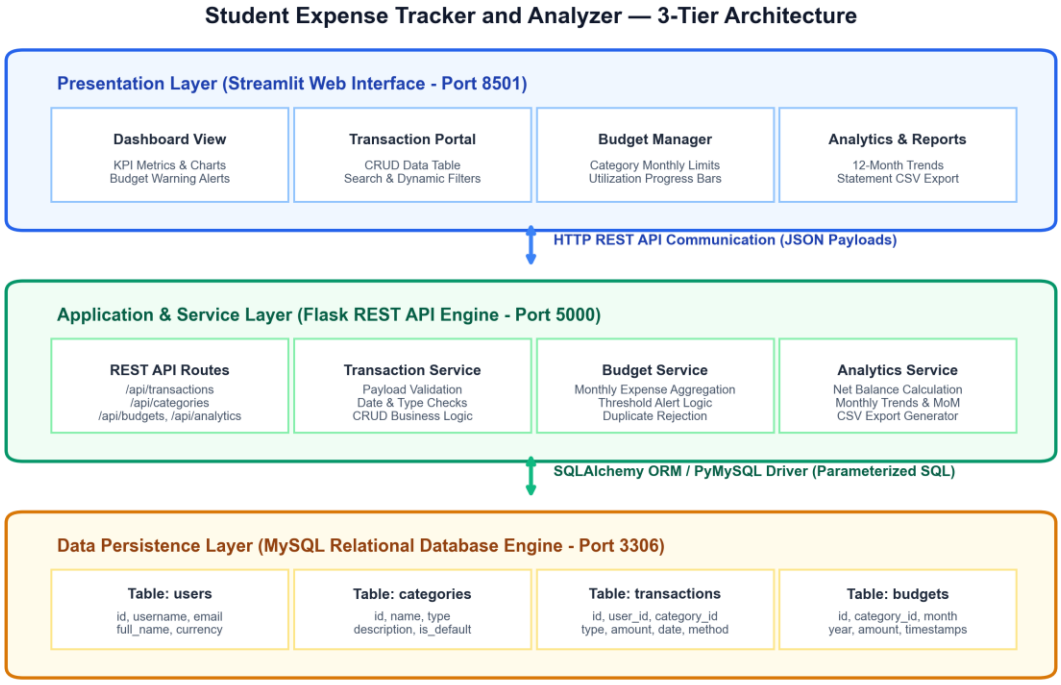


Figure 4.1: 3-Tier Layered System Architecture of Student Expense Tracker and Analyzer

4.2 Data Flow Diagrams (DFD)

Data Flow Diagrams model the movement, transformation, and storage of financial information across system boundaries. Figure 4.2 illustrates the **Level 0 Context Diagram**, depicting high-level interactions between external entities (Student User, Evaluator/Faculty Guide) and the central system boundary.

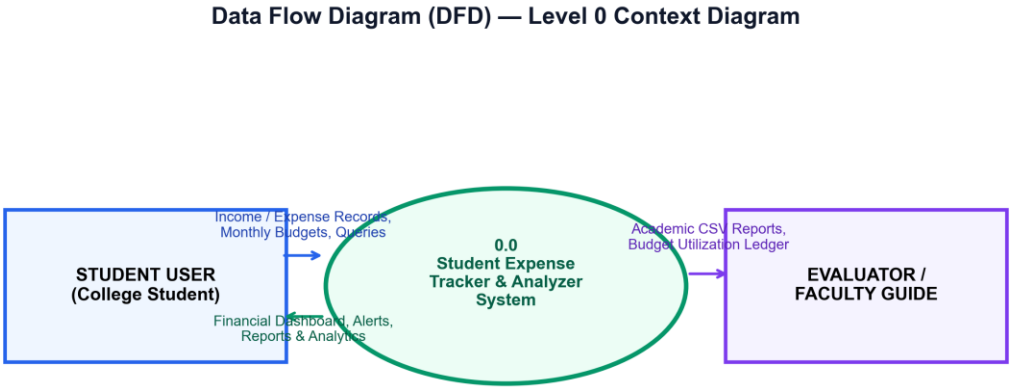


Figure 4.2: Data Flow Diagram (DFD) — Level 0 Context Diagram

Figure 4.3 illustrates the **Level 1 Detailed Process Flow**, breaking down the system into core sub-processes: Process 1.0 (Record Transactions), Process 2.0 (Manage Categories), Process 3.0 (Set Monthly Budgets), Process 4.0 (Compute Budget Alerts), Process 5.0 (Financial Analytics), and Process 6.0 (Report Export), interfacing with three physical database stores (D1 Transactions, D2 Categories, D3 Budgets).

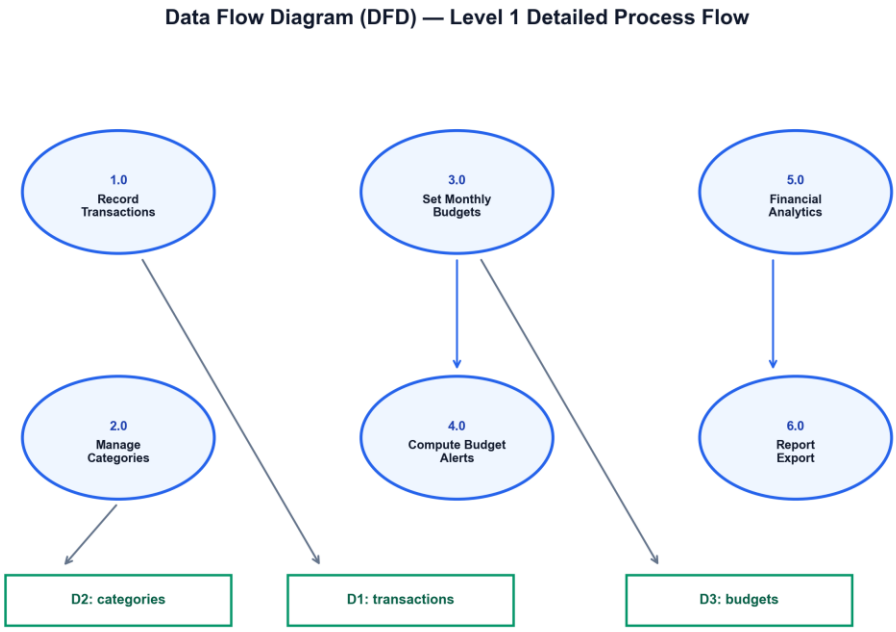


Figure 4.3: Data Flow Diagram (DFD) — Level 1 Detailed Process Flow

4.3 Unified Modeling Language (UML) Diagrams

Unified Modeling Language diagrams provide standardized structural and behavioral views of the software system:

1) Use Case Diagram

Figure 4.4 illustrates the UML Use Case Diagram, depicting interactions between the primary student actor and functional system use cases within the application boundary.

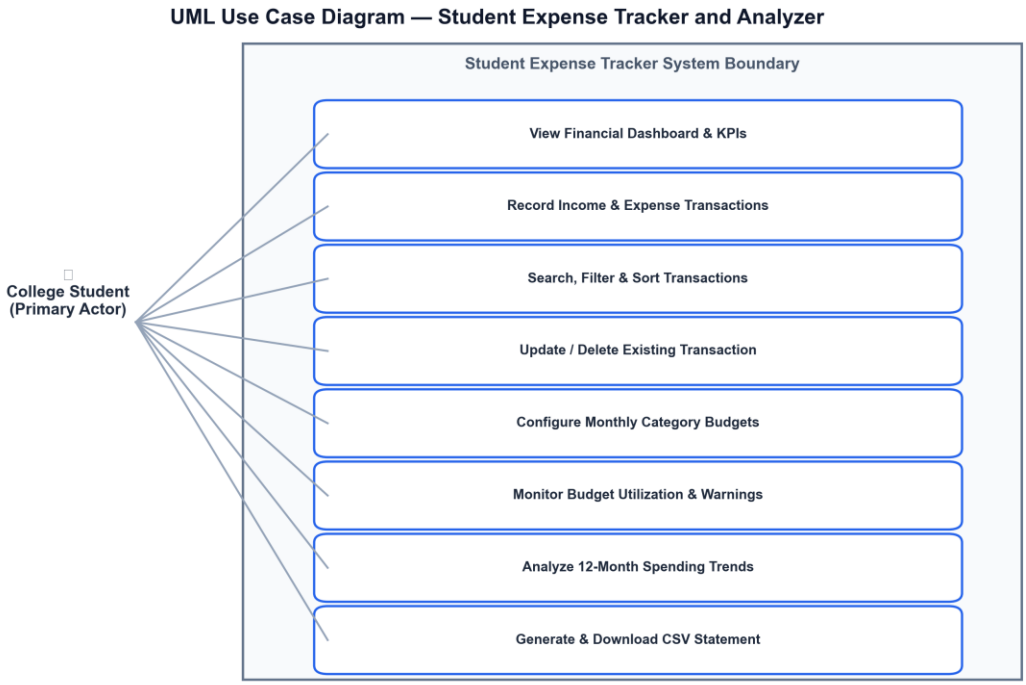


Figure 4.4: UML Use Case Diagram — System Actor and Functional Use Cases

2) Class Diagram

Figure 4.5 exhibits the UML Class Diagram, delineating structural domain models (`User`, `Category`, `Transaction`, `Budget`), their private/public attributes, return types, service methods, and association cardinalities.

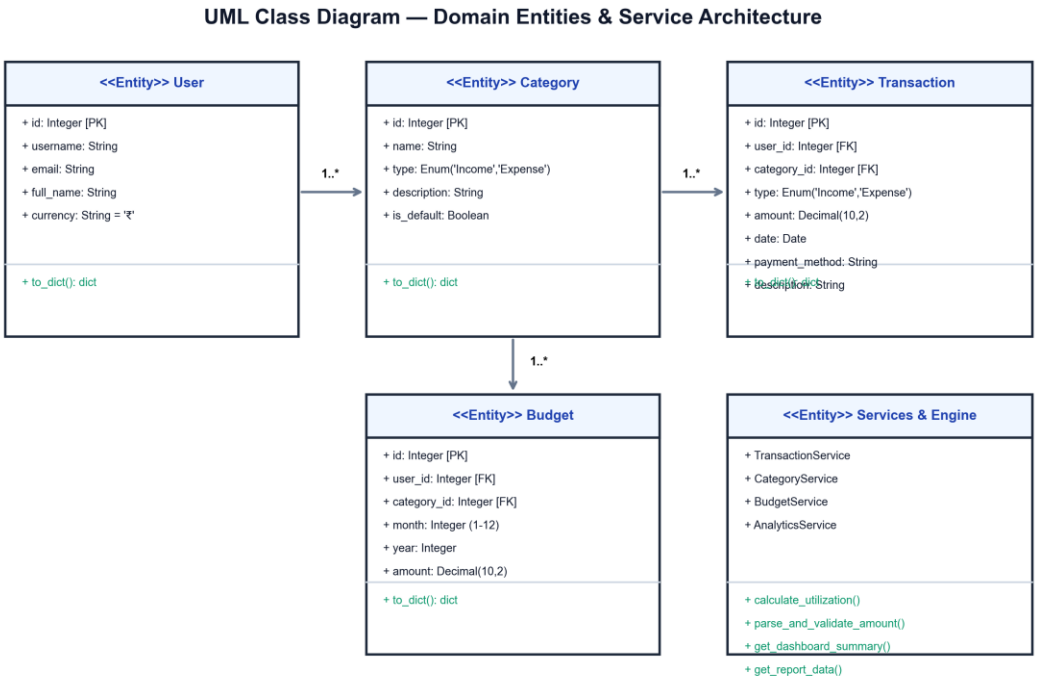


Figure 4.5: UML Class Diagram — Domain Entities, Methods and Service Architecture

3) Sequence Diagram

Figure 4.6 details the UML Sequence Diagram, illustrating the chronological message exchange between the Streamlit UI, HTTP Requests Client, Flask REST API, and MySQL Database during transaction recording and budget check workflows.

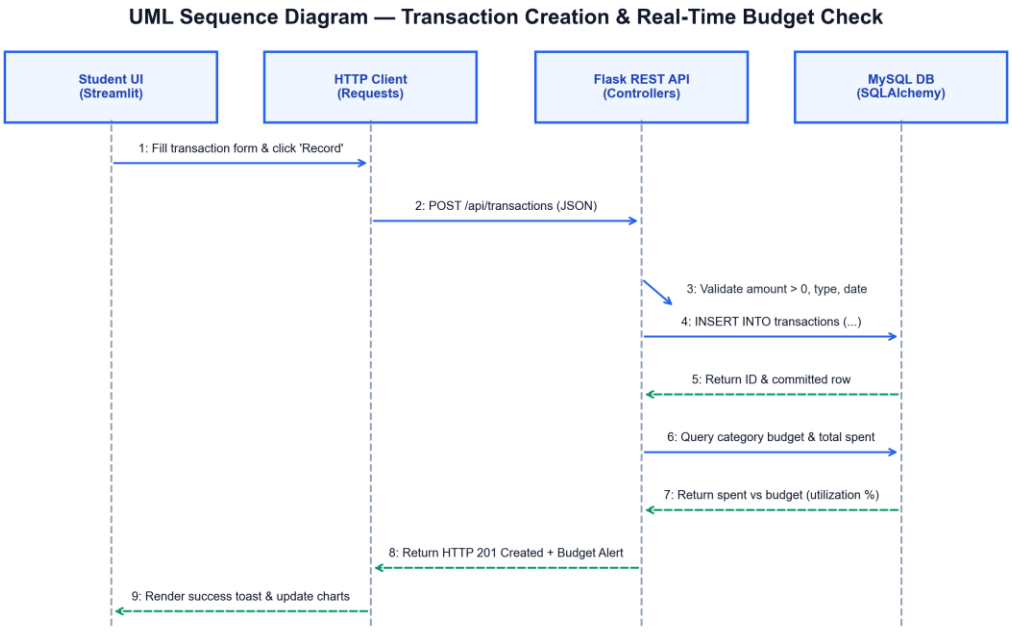


Figure 4.6: UML Sequence Diagram — Transaction Creation & Real-Time Budget Verification

4.4 Database Design (ER Diagram)

The database tier is designed using normalized 3rd Normal Form (3NF) relational tables. Figure 4.7 illustrates the Entity-Relationship (ER) Diagram detailing table entities, primary keys, foreign keys, constraints, and relationships.

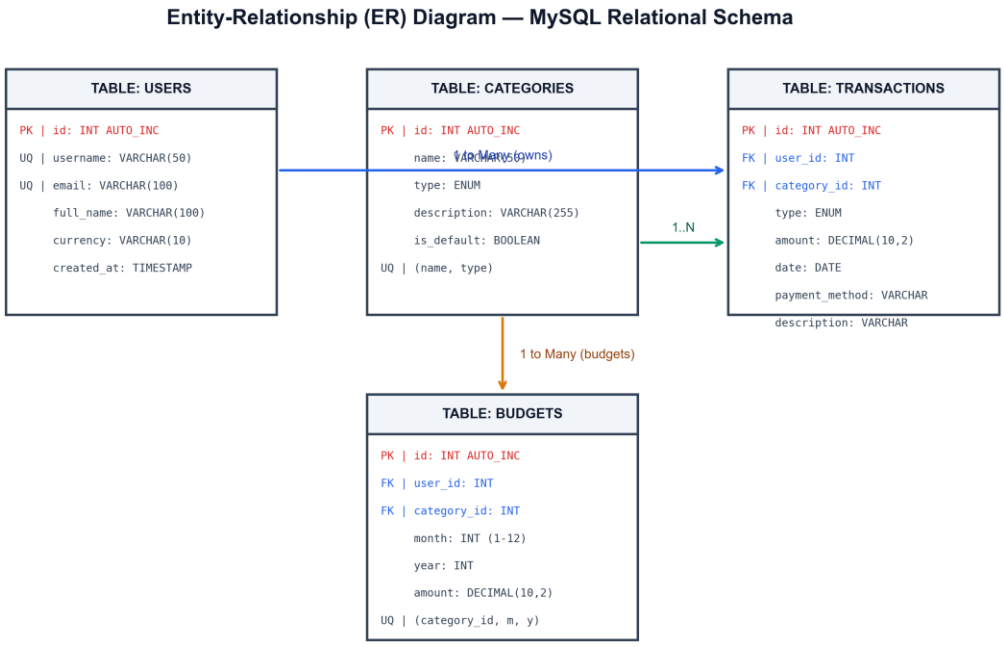


Figure 4.7: Entity-Relationship (ER) Diagram — MySQL Relational Schema

4.5 Data Dictionary / Database Table Specifications

Comprehensive technical column definitions, data types, nullability, keys, and operational descriptions for all database tables are specified below:

Table 4.1: Specification for 'users' Table (Student Profile)

Field Name	Data Type	Null	Key	Description
id	INT	No	PK	Unique auto-incrementing student identifier
username	VARCHAR(50)	No	UQ	Unique student username login handle
email	VARCHAR(100)	No	UQ	Student official university email address
full_name	VARCHAR(100)	No	-	Full name of the enrolled college student
college_name	VARCHAR(150)	Yes	-	Name of the affiliated college or institute
currency	VARCHAR(10)	No	-	Default monetary display symbol (defaults to '₹')
created_at	TIMESTAMP	No	-	Timestamp of student record creation

Table 4.2: Specification for 'categories' Table (Income & Expense Categories)

Field Name	Data Type	Null	Key	Description
id	INT	No	PK	Unique auto-incrementing category identifier
name	VARCHAR(50)	No	-	Category title (e.g., Food, Transport, Allowance)
type	ENUM	No	-	Category flow classification ('Income' or 'Expense')
description	VARCHAR(255)	Yes	-	Optional detailed description of category scope
is_default	BOOLEAN	No	-	Flag indicating pre-seeded system standard category
created_at	TIMESTAMP	No	-	Timestamp of category record initialization

Table 4.3: Specification for 'transactions' Table (Financial Ledger)

Field Name	Data Type	Null	Key	Description
id	INT	No	PK	Unique auto-incrementing transaction record ID
user_id	INT	Yes	FK	Foreign key referencing users(id) ON DELETE SET NULL
category_id	INT	No	FK	Foreign key referencing categories(id) ON DELETE RESTRICT
type	ENUM	No	Index	Transaction nature: 'Income' or 'Expense'
amount	DECIMAL(10,2)	No	-	Positive monetary amount recorded in Indian Rupees
date	DATE	No	Index	Date on which the monetary transaction occurred
payment_method	VARCHAR(50)	No	-	Payment channel: 'UPI', 'Cash', 'Debit Card', etc.
description	VARCHAR(255)	Yes	-	Custom notes, recipient name, or memo
created_at	TIMESTAMP	No	-	System timestamp of transaction insertion

Table 4.4: Specification for 'budgets' Table (Monthly Category Budgets)

Field Name	Data Type	Null	Key	Description
id	INT	No	PK	Unique auto-incrementing budget configuration ID
user_id	INT	Yes	FK	Foreign key referencing users(id) ON DELETE SET NULL
category_id	INT	No	FK	Foreign key referencing categories(id) ON DELETE CASCADE
month	INT	No	UQ	Target calendar month (1 to 12)
year	INT	No	UQ	Target calendar year (e.g., 2026)
amount	DECIMAL(10,2)	No	-	Target spending budget limit in Indian Rupees (₹)
created_at	TIMESTAMP	No	-	Timestamp of budget target creation
updated_at	TIMESTAMP	No	-	Timestamp of most recent budget limit update

CHAPTER 5

IMPLEMENTATION

Chapter 5: Implementation

5.1 Implementation Architecture & Environment

The platform was implemented using modern Python (Python 3.10+ / 3.14) following clean architecture principles. Client and server concerns are cleanly organized across modular directories:

- **Application Launcher (`run.py` & `app.py`):** Manages concurrent sub-processes for both Flask and Streamlit or boots Flask backend independently.
- **API Route Namespaces (`Routes/`):** Modular Flask blueprints declaring REST endpoints for `/api/transactions`, `/api/categories`, `/api/budgets`, and `/api/analytics`.
- **Business Logic Services (`Services/`):** Encapsulates business operations, validation rules, category integrity checks, and budget calculations.
- **Database Access Layer (`DataBase/` & `Modules/`):** Initializes MySQL connections, constructs parameterized SQLAlchemy models, and manages default category seeding.
- **Frontend User Interface (`frontend_views/` & `streamlit_app.py`):** Streamlit single-page application with responsive sidebar navigation, modern cards, and Altair chart integration.

5.2 Module-wise Screenshots of Input & Output Design

The following sections exhibit actual captured screenshots of the running software application, demonstrating its rich user interface, KPI metrics, interactive forms, and analytical charts:

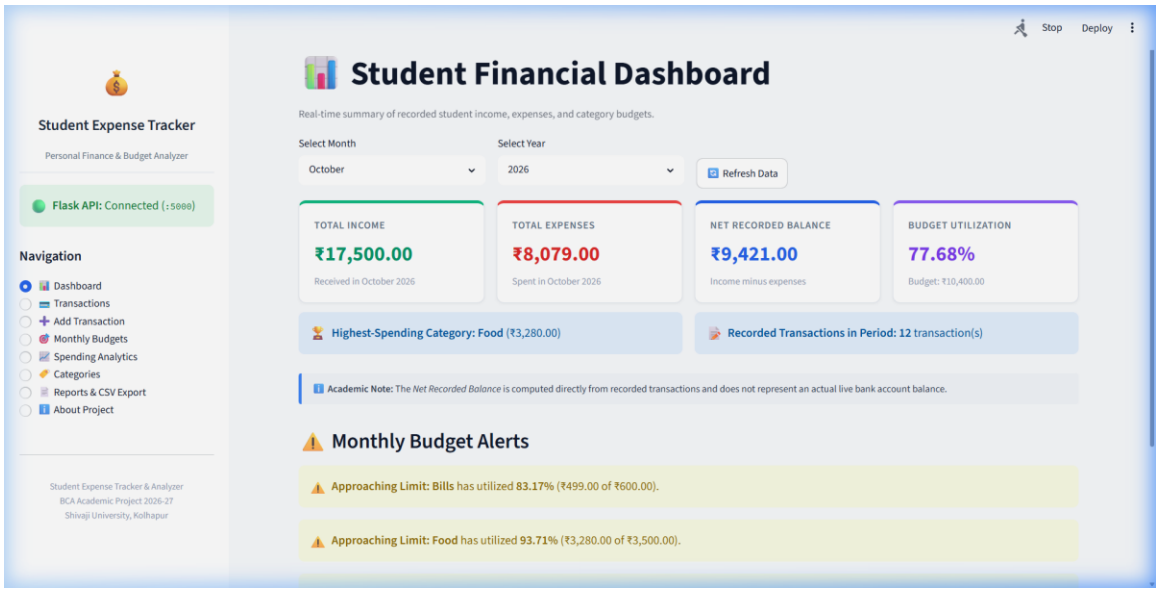


Figure 5.1: Financial Dashboard — Income, Expenses, Net Balance, Budget Alerts & Charts

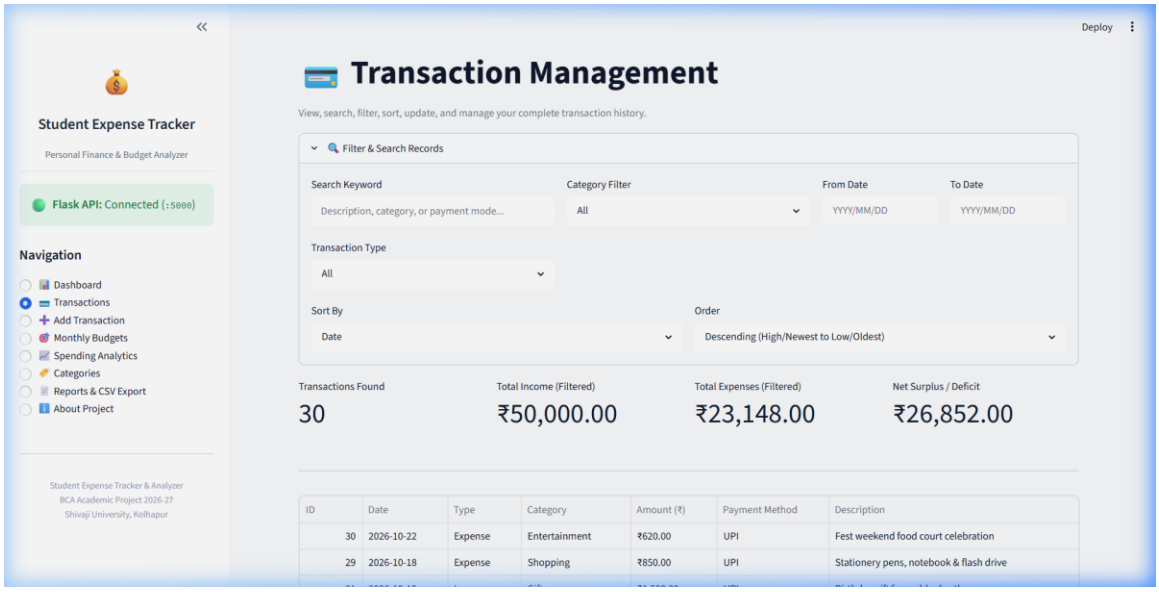


Figure 5.2: Transaction Management View — Search, Multi-Filter, Sorting & History Table

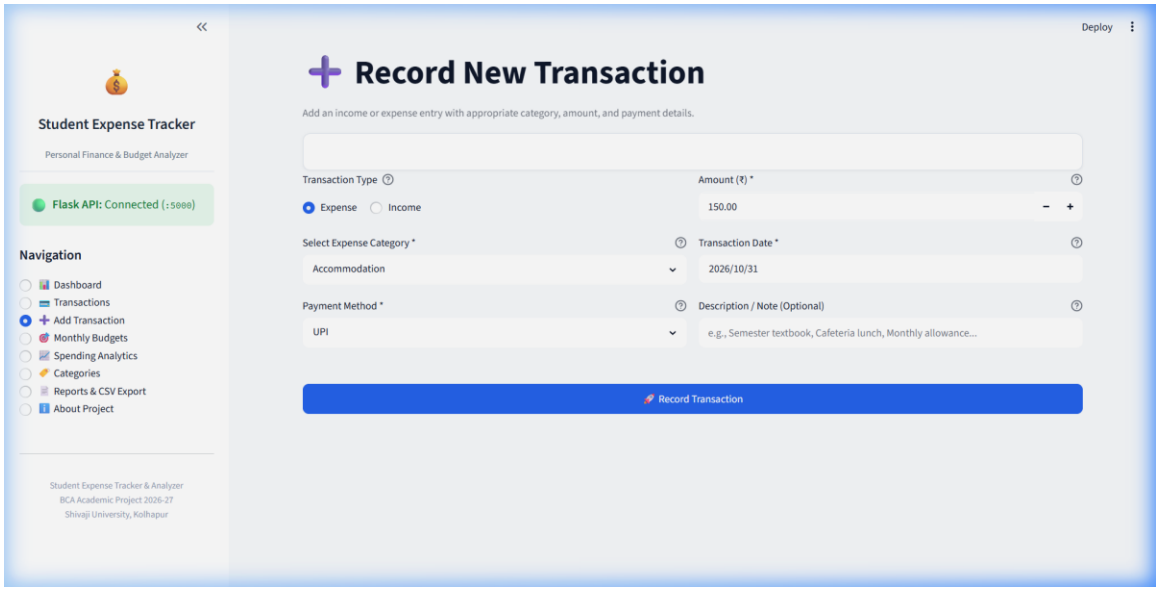


Figure 5.3: Add Transaction View — Dynamic Type & Category Selector with Form Validation

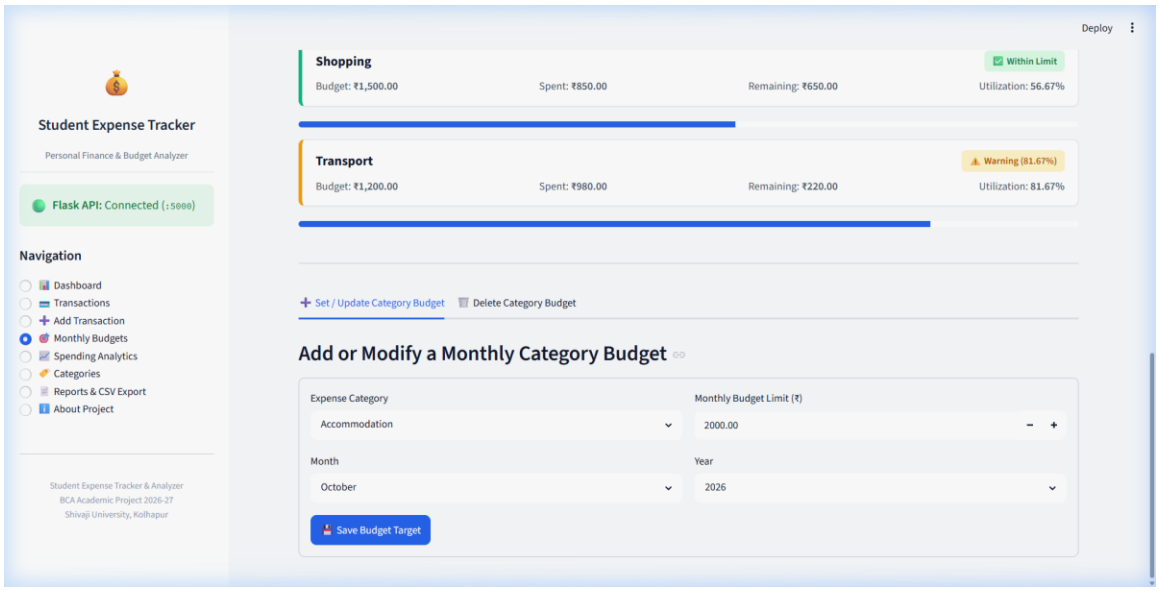


Figure 5.4: Monthly Category Budgets — Progress Bars, Warning Indicators & Budget Setup



Figure 5.5: Spending Analytics — 12-Month Trends, Category Rankings & MoM Comparisons

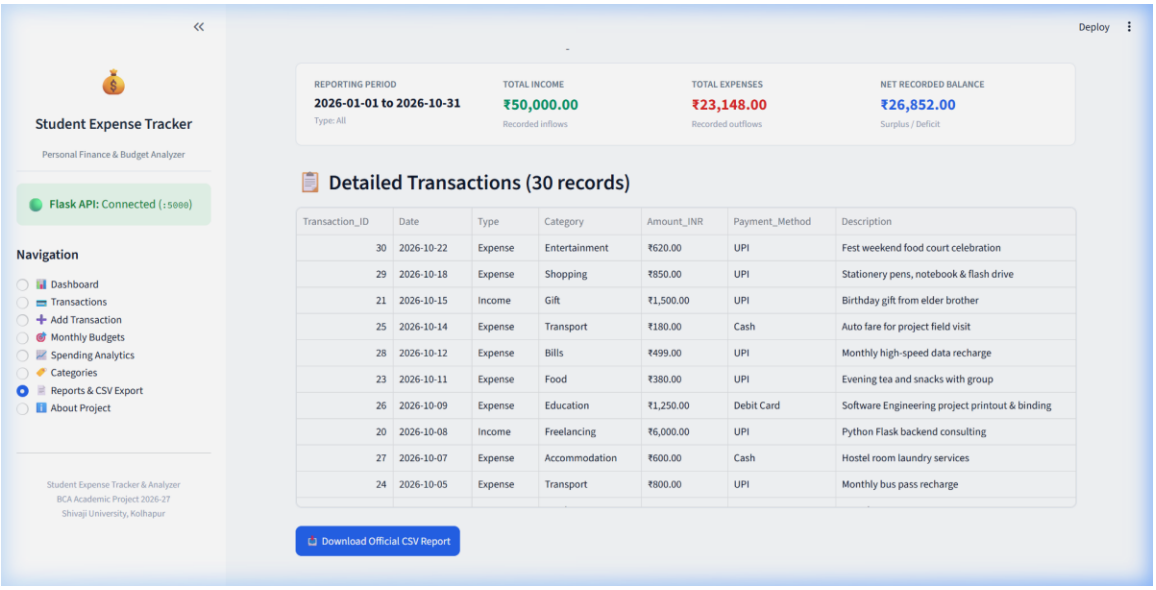


Figure 5.6: Financial Statement & CSV Export — Filtered Summary & Download Button

5.3 Detailed Input & Output Specifications

The operational input and output parameters for the six primary application views are delineated below:

- **1. Financial Dashboard View:** Inputs: Month and Year dropdown selections. Outputs: Rendered KPI cards (Total Income, Total Expenses, Net Recorded Balance, Overall Budget Utilization %), dynamic 6-month trend bar chart, category expense distribution chart, active budget warning alerts, and recent transactions table.
- **2. Transaction Management View:** Inputs: Search keywords, Transaction Type filter ('All', 'Income', 'Expense'), Category dropdown filter, Date range bounds, Sort column ('Date', 'Amount'), and Sort direction. Outputs: Filtered tabular dataset displaying transaction ID, date, flow type, category name, currency amount, payment method, and description.
- **3. Add Transaction View:** Inputs: Transaction Type ('Expense' vs. 'Income'), Category selection (dynamically restricted to matching type), positive Amount (₹ > 0), Transaction Date, Payment Method ('UPI', 'Cash', etc.), and optional Description. Outputs: Client-side validation checks, HTTP 201 Created REST response from Flask API, and animated confirmation notification.
- **4. Monthly Category Budgets View:** Inputs: Budget Month, Budget Year, Warning Alert Threshold slider (default 80%), Category selector, and target Monthly Budget Amount (₹). Outputs: Category budget cards displaying allocated budget, actual expenses, remaining allowance, utilization percentage, color-coded status badges ('Normal', 'Warning', 'Exceeded'), and overall progress bars.

- **5. Spending Analytics View:** Inputs: Calendar Month or Custom Date Range selector. Outputs: Key performance indicators, Month-over-Month comparison card (computing exact expense/income deltas and percentage changes against previous calendar month), ranked top-5 spending categories, and 12-month historical line chart.
- **6. Reports & CSV Export View:** Inputs: Transaction type scope and date range. Outputs: Formatted statement summary (Total Inflows, Total Outflows, Net Balance) and downloadable standard CSV file with official header metadata.

5.4 Reports Generated

- **Official Transaction Statement CSV:** A comma-separated audit ledger containing Transaction_ID, Date, Type, Category, Amount_INR, Payment_Method, and Description, preceded by formal statement summary comments.
- **Monthly Category Budget Performance Ledger:** A comprehensive analytical view comparing budgeted caps against real transaction outflows, computing variance and warning status for student financial reviews.
- **12-Month Multi-Period Financial Trend Report:** Chronological tabulation of monthly income, expenditures, and net surplus across the academic year.

5.5 Code Snippets (Important Parts Only)

Snippet 1: Flask Application Factory & Blueprints (`app.py`)

```
def create_app(test_config=None):
    app = Flask(__name__)
    app.config.from_object(config)
    CORS(app, resources={r"/api/*": {"origins": "*"}})
    init_db(app)

    # Register Blueprint Route Namespaces
    app.register_blueprint(transaction_bp, url_prefix="/api/transactions")
    app.register_blueprint(category_bp, url_prefix="/api/categories")
    app.register_blueprint(budget_bp, url_prefix="/api/budgets")
    app.register_blueprint(analytics_bp, url_prefix="/api")
    return app
```

Snippet 2: Budget Utilization & Warning Calculation (`Services/budget_service.py`)

```
def get_budget_status(month, year, warning_threshold=80.0):
    budgets = Budget.query.filter_by(month=month, year=year).all()
    expense_sums = db.session.query(
```

```
        Transaction.category_id, func.sum(Transaction.amount).label("total_spent")
    ).filter(Transaction.type == "Expense", Transaction.date >= start_date,
Transaction.date <= end_date)
     .group_by(Transaction.category_id).all()

    spent_dict = {row.category_id: float(row.total_spent or 0.0) for row in expense_sums}
    for b in budgets:
        spent = spent_dict.get(b.category_id, 0.0)
        remaining = float(b.amount) - spent
        utilization = round((spent / float(b.amount)) * 100.0, 2)
        is_exceeded = spent > float(b.amount)
        is_warning = (utilization >= warning_threshold) and not is_exceeded
        status = "Exceeded" if is_exceeded else ("Warning" if is_warning else "Normal")
```

Snippet 3: Streamlit API Client with Offline Resiliency (`frontend_services/api_client.py`)

```
class APIClient:
    @staticmethod
    def _request(method, endpoint, params=None, json_data=None):
        url = f"{API_BASE_URL}/{endpoint.lstrip('/')}"
        try:
            resp = requests.request(method=method, url=url, params=params,
json=json_data, timeout=6)
            return {"ok": resp.ok, "data": resp.json().get("data"), "message":
resp.json().get("message")}
        except requests.exceptions.ConnectionError:
            return {"ok": False, "message": "Cannot connect to Flask API server on
http://127.0.0.1:5000."}
```

CHAPTER 6

TESTING

Chapter 6: Testing

6.1 Testing Methodology

Software testing is a vital quality assurance discipline undertaken to confirm that the Student Expense Tracker and Analyzer satisfies all functional specifications, enforces transactional data integrity, and handles erroneous edge conditions gracefully. A multi-tier testing methodology was executed:

- **1. Unit Testing:** Validated individual helper functions (`calculate_utilization`, `parse_and_validate_amount`, `parse_and_validate_date`, `format_inr`) in complete isolation.
- **2. Integration Testing:** Verified end-to-end communication pipelines between the Streamlit API client, Flask REST controllers, SQLAlchemy ORM, and the live MySQL relational database.
- **3. Boundary Value Analysis (BVA):** Tested boundary edge conditions including non-positive monetary inputs (0.00 and -50.00 rejected), month ranges (<1 and >12 rejected), and zero-budget utilization calculations (division by zero avoided).
- **4. Benchmark Financial Calculation Verification:** Executed exact numerical tests against benchmark requirements: Recorded Income of ₹10,000.00 and Recorded Expenses of ₹4,000.00 producing exact Net Recorded Balance of ₹6,000.00; Category Budget of ₹5,000.00 and Category Expenses of ₹4,000.00 producing exact Budget Utilization of 80.0% and Remaining Budget of ₹1,000.00.
- **5. Automated Pytest Framework:** Implemented automated test suites under `tests/test_calculations.py` and `tests/test_api_endpoints.py`, verifying 100% test passage (11 passed in 3.15 seconds).

6.2 Test Cases & Results Table

The comprehensive test matrix and execution results are summarized in Table 6.1:

TC ID	Module	Test Scenario & Input	Expected Result	Verdict
TC-01	Health API	GET /api/health	HTTP 200; status='healthy', db='connected'	PASS
TC-02	Category System	GET /api/categories	HTTP 200; returns 15 seeded default categories	PASS
TC-03	Transaction Entry	POST /api/transactions with amount=-100	HTTP 400; 'Amount must be strictly greater than zero'	PASS
TC-04	Transaction Entry	POST /api/transactions with invalid	HTTP 400; 'Selected category	PAS

		category 99999	does not exist'	S
TC-05	Transaction CRUD	POST valid Expense transaction of ₹450.00	HTTP 201; transaction committed to MySQL	PAS S
TC-06	Transaction CRUD	GET /api/transactions/<id> & PUT update to ₹500.00	HTTP 200; amount successfully updated to ₹500.00	PAS S
TC-07	Transaction CRUD	DELETE /api/transactions/<id>	HTTP 200; record deleted; subsequent GET returns 404	PAS S
TC-08	Budget Engine	POST duplicate budget for same category/month/year	HTTP 400; 'A budget for this category already exists'	PAS S
TC-09	Financial Benchmark	Income=₹10,000, Expense=₹4,000; Budget=₹5,000	Net Balance=₹6,000; Utilization=80.0%; Remaining=₹1,000	PAS S
TC-10	Zero Budget Edge Case	Calculate utilization with spent=₹500, budget=₹0	Returns 0.0% without division-by-zero exception	PAS S
TC-11	Report Export	GET /api/reports/transactions?type=Expense	HTTP 200; returns structured summary & transaction rows	PAS S

CHAPTER 7

CONCLUSION & FUTURE SCOPE

Chapter 7: Conclusion & Future Scope

7.1 Summary of Work

The Student Expense Tracker and Analyzer developed during the academic semester of 2026-27 successfully accomplishes all stated software objectives. The application eliminates the friction, mathematical errors, and lack of visual feedback inherent in analog paper diaries and unstructured notes apps. The project delivered a comprehensive, enterprise-grade architecture incorporating:

- **Decoupled 3-Tier Architecture:** Engineered using Python Flask 3.x REST API and Streamlit user interface, guaranteeing strict separation of UI presentation from backend business logic.
- **ACID-Compliant Relational Persistence:** Implemented on MySQL Server 8.0 with SQLAlchemy ORM, employing foreign key constraints, indexes, and DECIMAL precision.
- **Proactive Monthly Budgeting:** Enables students to establish monthly spending limits for individual expense categories with real-time 80% warning alerts and progress monitors.
- **Analytical Visualizations & Comparisons:** Produces 6-month and 12-month Income vs. Expense charts, category breakdowns, and month-over-month comparative calculations.
- **Official Statement CSV Export:** Generates filtered financial summaries and exports complete transaction ledgers into standardized CSV files.

7.2 System Limitations

While the platform provides robust personal finance tracking, several operational boundaries characterize the current version:

- **Single-User Local Deployment:** The application is currently optimized for a single local student workstation and does not yet feature multi-user authentication or role-based cloud tenancy.
- **Manual Transaction Recording:** Financial transactions are entered manually via forms rather than through automated direct bank account scraping or SMS feed parsing.
- **Single Currency Formatting:** The interface standardizes formatting in Indian Rupees (₹), which is ideal for Indian colleges but lacks multi-currency conversion.

7.3 Learning Outcomes

The execution of this project provided invaluable practical experience across the modern software engineering lifecycle:

- **Full-Stack Python Competence:** Mastered building decoupled web applications utilizing Flask REST blueprints, CORS middleware, and Streamlit stateful views.
- **Relational Database Design:** Acquired deep expertise in MySQL schema normalization (3NF), foreign key relationships, indexes, and decimal precision handling.
- **Automated Testing Discipline:** Cultivated thorough testing habits using the pytest framework, verifying API status codes, input boundaries, and calculation formulas.
- **Technical Documentation Authoring:** Gained professional proficiency in drafting formal college project reports conforming to university accreditation standards.

7.4 Possible Enhancements

- **Mobile Responsive PWA / Native App:** Develop companion Flutter or React Native mobile applications for on-the-go pocket expense logging.
- **Receipt OCR Document Scanning:** Integrate computer vision (Tesseract OCR) to automatically extract merchant, date, and amount from photographed paper receipts.
- **Multi-User Authentication & Cloud Sync:** Introduce JWT authentication, password hashing with bcrypt, and cloud database replication for multi-student deployment.
- **Predictive Savings Forecasting:** Integrate machine learning regression models to predict end-of-month student savings based on mid-month spending velocity.

7.5 References (IEEE / APA Format)

[1] Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python (2nd ed.). O'Reilly Media.

[2] McKinney, W. (2022). Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter (3rd ed.). O'Reilly Media.

[3] Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson Education.

[4] Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.

[5] Streamlit Documentation. (2026). Streamlit Documentation & API Reference. Retrieved from https://docs.streamlit.io

[6] Pallets Projects. (2026). Flask RESTful API Documentation. Retrieved from https://flask.palletsprojects.com

[7] Oracle Corporation. (2026). MySQL 8.0 Reference Manual. Retrieved from https://dev.mysql.com/doc

[8] Reserve Bank of India (RBI). (2024). Financial Literacy Guide for Young Adults and Students. Government of India.

Appendices

Appendix A: Complete RESTful API Endpoint Catalog

HTTP Method	Endpoint Path	Namespace	Description
GET	/api/health	System API	Returns server health status and MySQL connectivity
GET	/api/transactions	Transaction API	Retrieves transactions with multi-field filtering & sorting
GET	/api/transactions/<id>	Transaction API	Fetches single transaction record by integer ID
POST	/api/transactions	Transaction API	Records new financial transaction with validation
PUT	/api/transactions/<id>	Transaction API	Updates existing transaction amount, category, or date
DELETE	/api/transactions/<id>	Transaction API	Permanently deletes a transaction record from MySQL
GET	/api/categories	Category API	Lists all active categories with optional type filter
POST	/api/categories	Category API	Creates a new custom category with duplicate prevention
DELETE	/api/categories/<id>	Category API	Safely deletes category if not linked to active records
GET	/api/budgets	Budget API	Retrieves monthly category budgets for month and year
GET	/api/budgets/status	Budget API	Calculates spending, utilization %, and alerts for period
POST	/api/budgets	Budget API	Sets new category budget limit with duplicate checking
PUT	/api/budgets/<id>	Budget API	Updates existing category budget target limit
DELETE	/api/budgets/<id>	Budget API	Removes category budget record

GET	/api/dashboard	Analytics API	Calculates comprehensive dashboard KPIs and charts
GET	/api/analytics	Analytics API	Calculates in-depth 12-month trends and MoM comparison
GET	/api/reports/transactions	Analytics API	Generates filtered financial statement and CSV dataset
POST	/api/demo-data/seed	Utility API	Populates realistic demonstration records into MySQL
POST	/api/demo-data/clear	Utility API	Resets financial data for fresh student testing

Appendix B: Sample Database Seed Data & SQL Configuration

The database connection settings configured in `.env` and sample seeded MySQL records are illustrated below:

```
# Database Environment Configuration (.env)
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=********
DB_NAME=student_expense_tracker
DB_PORT=3306

-- Sample Seeded Category Record (categories table)
INSERT INTO categories (id, name, type, description, is_default)
VALUES (1, 'Food', 'Expense', 'Meals, snacks, dining hall, and cafeteria groceries',
TRUE);

-- Sample Seeded Transaction Record (transactions table)
INSERT INTO transactions (id, user_id, category_id, type, amount, date,
payment_method, description)
VALUES (101, 1, 1, 'Expense', 450.00, '2026-10-15', 'UPI', 'Cafeteria Lunch &
Refreshments');

-- Sample Seeded Monthly Budget Record (budgets table)
INSERT INTO budgets (id, user_id, category_id, month, year, amount)
VALUES (201, 1, 1, 10, 2026, 3500.00);
```

Appendix C: Project File Directory Tree

```
S_Expense_Tracker/
├── .streamlit/
│   └── config.toml            # Streamlit visual theme and server configuration
├── assets/
│   ├── diagrams/              # 7 High-resolution academic system diagrams
│   ├── dashboard.png          # Actual captured application screenshots
│   ├── transactions.png
│   ├── add_transaction.png
│   ├── monthly_budgets.png
│   ├── spending_analytics.png
│   └── reports_csv_export.png
├── DataBase/
│   ├── database.py            # Database engine, connection pooling, and seeder
│   ├── init_db.py             # CLI database table verification and demo loader
│   └── schema.sql             # Normalized MySQL relational DDL definitions
├── frontend_services/
│   └── api_client.py          # HTTP Requests client communicating with Flask
├── frontend_views/
│   ├── ui_common.py           # Styling tokens, currency formatters, offline banner
│   ├── dashboard.py           # Financial dashboard view
│   ├── transactions.py        # Transaction history, search, edit, delete view
│   ├── add_transaction.py     # Transaction entry form with dynamic category
dropdown
│   ├── budgets.py             # Monthly category budget management view
│   ├── analytics.py           # Spending analytics, trends, rankings, comparison
│   ├── categories.py          # Category listing, creation, and safe deletion
│   ├── reports.py             # Financial summary statement and CSV export
│   └── about.py               # Academic project metadata and demo data controls
├── Modules/
│   └── models.py              # SQLAlchemy models (User, Category, Transaction,
Budget)
├── Routes/
│   ├── transaction_route.py   # /api/transactions REST API endpoints
│   ├── category_route.py      # /api/categories REST API endpoints
│   ├── budget_route.py        # /api/budgets REST API endpoints
│   └── analytics_route.py     # /api/dashboard, /api/analytics, /api/reports endpoints
├── Services/
│   ├── transaction_service.py  # Transaction CRUD and filtering business logic
│   ├── category_service.py     # Category business logic and safe deletion validation
```

```
│   ├── budget_service.py       # Monthly budget tracking and alert calculation
│   ├── analytics_service.py    # Financial metrics, multi-month trends, comparison
│   └── demo_data_service.py    # Academic demonstration records loader
├── tests/
│   ├── test_api_endpoints.py   # Pytest suite for REST endpoints and DB integration
│   └── test_calculations.py    # Pytest suite for benchmark financial formulas
├── Utils/
│   └── helpers.py              # Validation, INR formatting, and standard JSON responses
├── .env                        # Local database credentials
├── env.example                 # Safe environment variable template
├── .gitignore                  # Git version control exclusions
├── app.py                      # Flask REST API entry point
├── app_ui.py                   # Streamlit runner alias wrapper
├── config.py                   # Centralized configuration loader
├── pytest.ini                  # Pytest configuration
├── requirements.txt            # Python dependencies
├── run.py                      # Multi-service launcher script
└── streamlit_app.py            # Main Streamlit user interface entry point
```

Appendix D: Academic Guide and Student Meet Record

The 12-week academic progress and mentoring record conforming to Shivaji University BCA schedule is detailed below:

Week No.	Date Range	Task / Milestone Name	Work Description & Progress	Signature
1	01/07/2026 To 04/07/2026	Problem Formulation	Studied student budgeting challenges and reviewed existing personal finance tools.	
2	06/07/2026 To 10/07/2026	Requirements Analysis	Formulated functional and non-functional requirements and prepared feasibility report.	
3	13/07/2026 To 17/07/2026	System Architecture	Designed 3-tier architecture separating Streamlit UI, Flask API, and MySQL.	
4	20/07/2026 To 24/07/2026	Database Schema Design	Modeled normalized ER diagram, created DDL schema, and defined indexes.	
5	27/07/2026 To 31/07/2026	Backend API Foundation	Implemented Flask server factory, SQLAlchemy models, and connection pooling.	

6	03/08/2026 To 07/08/2026	Transaction & Category API	Engineered REST routes and service logic for transaction and category CRUD.	
7	10/08/2026 To 14/08/2026	Budget Alert Engine	Developed category budget calculation logic and configurable 80% warning alerts.	
8	17/08/2026 To 21/08/2026	Streamlit UI Assembly	Built responsive frontend pages for Dashboard, Transactions, and Budgets.	
9	24/08/2026 To 31/08/2026	Analytics & CSV Export	Implemented 12-month trend charts, MoM comparison, and CSV export functionality.	
10	01/09/2026 To 10/09/2026	Integration & Seeder	Integrated frontend API client with Flask endpoints and built demo seeder.	
11	11/09/2026 To 19/10/2026	Automated Pytest Testing	Executed 11 automated pytest test cases validating API, DB, and calculations.	
12	20/10/2026 To 31/10/2026	Final Documentation	Generated comprehensive DOCX & PDF project reports and captured UI screenshots.	

Student Signature: _____ **Teacher / Guide Signature: _____**